Pandas Master Cheat Sheet

Beginner → Advanced • Analytics • Data Engineering • Interview Reference

How to use: keep this beside your notebook while practicing on your pandas_practice_dataset.csv. Focus on the problem a method solves, not memorization.

This cheat sheet follows the comprehensive Pandas study plan you provided: Series/DataFrame, I/O, inspection, selection, filtering, cleaning, GroupBy, joins, reshaping, window-like operations, validation, ETL, SQL/PySpark comparisons, performance, analytics, and interview patterns.

1. Core Mental Model

Series = one-dimensional labeled array (commonly one column). **DataFrame** = two-dimensional labeled table made of aligned Series.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({"name":["John","Alex"], "age":[25,30]})
df["age"] # Series
df[["age"]] # DataFrame
```

Method / Pattern	What it does	Example
pd.Series()	Create a Series.	`pd.Series([10, 20, 30])`
pd.DataFrame()	Create a DataFrame.	`pd.DataFrame({"a":[1,2]})`
df.index	Row labels.	`df.index`
df.columns	Column labels.	`df.columns`
df.values	Underlying values representation.	`df.values`
df.ndim	Number of dimensions.	`df.ndim`
df.shape	Rows × columns.	`df.shape`

2. Creating & Reading Data

Method / Pattern	What it does	Example
pd.DataFrame(dict)	Create from dictionary.	`pd.DataFrame({"a":[1,2],"b":[3,4]})`
pd.DataFrame(list_of_dicts)	Create records-style table.	`pd.DataFrame([{"a":1},{"a":2}])`
pd.DataFrame(array)	Create from NumPy array.	`pd.DataFrame(np.array([[1,2],[3,4]]))`
pd.Series()	Create one-dimensional data.	`pd.Series([1,2,3])`
pd.read_csv()	Read CSV.	`pd.read_csv("data.csv")`
pd.read_excel()	Read Excel.	`pd.read_excel("data.xlsx")`
pd.read_json()	Read JSON.	`pd.read_json("data.json")`
pd.read_sql()	Read SQL query/table into DataFrame.	`pd.read_sql("SELECT * FROM t", con)`
pd.read_parquet()	Read Parquet.	`pd.read_parquet("data.parquet")`
pd.json_normalize()	Flatten nested JSON.	`pd.json_normalize(records)`

Important read_csv parameters

sep, **header**, **index_col**, **usecols**, **dtype**, **parse_dates**, **na_values**, **encoding**, **nrows**, **chunksize**.

Writing data

Method / Pattern	What it does	Example
df.to_csv()	Write CSV.	`df.to_csv("out.csv", index=False)`
df.to_excel()	Write Excel.	`df.to_excel("out.xlsx", index=False)`
df.to_json()	Write JSON.	`df.to_json("out.json", orient="records")`
df.to_sql()	Write to database.	`df.to_sql("table", con, if_exists="replace")`
df.to_parquet()	Write columnar Parquet.	`df.to_parquet("out.parquet", index=False)`

3. Inspection / Profiling

Method / Pattern	What it does	Example
df.head(n)	First n rows.	`df.head(5)`
df.tail(n)	Last n rows.	`df.tail(5)`
df.sample()	Random sample.	`df.sample(10, random_state=42)`
df.shape	Dimensions.	`df.shape`
df.size	Total cells.	`df.size`
df.columns	Column names.	`df.columns`
df.index	Index labels.	`df.index`
df.dtypes	Column data types.	`df.dtypes`
df.info()	Compact schema/null/memory summary.	`df.info()`
df.describe()	Descriptive statistics.	`df.describe()`
df.nunique()	Unique count per column.	`df.nunique()`
df.value_counts()	Frequency of row/value combinations.	`df.value_counts()`
df.memory_usage()	Memory usage.	`df.memory_usage(deep=True)`
df.empty	Whether DataFrame has no rows.	`df.empty`
df.select_dtypes()	Select columns by dtype.	`df.select_dtypes(include="number")`

First-pass workflow: shape → columns → dtypes/info → missingness → duplicates → describe → categorical distributions → date ranges → domain validation.

4. Selecting Rows & Columns

Method / Pattern	What it does	Example
df["col"]	One column → Series.	`df["revenue"]`
df[["a","b"]]	Multiple columns → DataFrame.	`df[["customer_id","revenue"]]`
df.loc[]	Label-based selection; supports boolean masks.	`df.loc[df["age"] > 30, ["name","age"]]`
df.iloc[]	Integer-position-based selection.	`df.iloc[0:5, 1:4]`
df.at[]	Fast scalar label access.	`df.at[0, "age"]`
df.iat[]	Fast scalar positional access.	`df.iat[0, 1]`
isin()	Membership filter.	`df[df["country"].isin(["India","USA"])]`
between()	Inclusive range filter by default.	`df[df["age"].between(18, 60)]`
query()	Expression-based row filtering.	`df.query("age > 30 and revenue > 10000")`

loc vs iloc: loc uses labels; **iloc** uses integer positions. For `loc`, boolean masks and label slices are natural. For `iloc`, positions are integers.

```
df.loc[2, "age"] # label 2, column age
df.iloc[2, 1] # third row, second column
```

5. Adding, Modifying, Renaming & Dropping

Method / Pattern	What it does	Example
df["new"] = ...	Create/overwrite a column.	`df["total"] = df["price"] * df["qty"]`
df.assign()	Create columns while returning a new DataFrame.	`df.assign(total=df["price"]*df["qty"])`
df.loc[mask, col] =	Conditional modification.	`df.loc[df["age"] < 18, "segment"] = "Minor"`
df.iloc[row, col] =	Positional modification.	`df.iloc[0, 1] = 99`
df.rename()	Rename rows/columns.	`df.rename(columns={"old":"new"})`
df.drop()	Drop rows/columns.	`df.drop(columns=["temp"])`
df.insert()	Insert a column at a position.	`df.insert(1, "segment", "A")`
df.pop()	Remove and return a column.	`revenue = df.pop("revenue")`
df.set_axis()	Set labels.	`df.set_axis(["a","b"], axis="columns")`

6. Missing Values, Duplicates & Data Types

Method / Pattern	What it does	Example
isna()/isnull()	Detect missing values.	`df.isna().sum()`
notna()/notnull()	Detect non-missing values.	`df.notna()`
dropna()	Remove rows/columns with missing values.	`df.dropna(subset=["customer_id"])`
fillna()	Replace missing values.	`df["revenue"].fillna(df["revenue"].median())`
ffill()	Forward-fill from previous value.	`df["status"].ffill()`
bfill()	Back-fill from next value.	`df["status"].bfill()`
duplicated()	Identify duplicate rows.	`df.duplicated()`
drop_duplicates()	Remove duplicate rows.	`df.drop_duplicates(subset=["customer_id"])`
astype()	Explicit dtype conversion.	`df["age"].astype("int64")`
pd.to_numeric()	Convert values to numeric; errors can be coerced.	`pd.to_numeric(s, errors="coerce")`
pd.to_datetime()	Convert to datetime.	`pd.to_datetime(df["onboarding_date"])`
convert_dtypes()	Infer modern nullable dtypes.	`df.convert_dtypes()`

Imputation rule: never fill nulls blindly. Decide whether null means unknown, not applicable, not yet observed, or data-quality failure; choose an imputation strategy based on business meaning.

7. String Operations

Method / Pattern	What it does	Example
.str.lower()	Lowercase.	`df["name"].str.lower()`
.str.upper()	Uppercase.	`df["country"].str.upper()`
.str.title()	Title case.	`df["name"].str.title()`
.str.strip()	Trim whitespace.	`df["name"].str.strip()`
.str.replace()	Replace text/patterns.	`df["email"].str.replace(" ", "", regex=False)`
.str.contains()	Substring/regex test.	`df["email"].str.contains("@example.com", na=False)`
.str.startswith()	Prefix test.	`df["id"].str.startswith("CUS")`
.str.endswith()	Suffix test.	`df["email"].str.endswith(".com", na=False)`
.str.len()	String length.	`df["name"].str.len()`
.str.split()	Split strings.	`df["name"].str.split(" ")`
.str.extract()	Extract regex groups.	`df["email"].str.extract(r"@(.+)$")`
.str.slice()	Slice strings.	`df["id"].str.slice(0, 3)`
.str.match()	Regex match from start.	`df["id"].str.match(r"^CUS")`

8. Date & Time

Method / Pattern	What it does	Example
pd.to_datetime()	Parse timestamps.	`df["date"] = pd.to_datetime(df["date"])`
.dt.year	Extract year.	`df["date"].dt.year`
.dt.month	Extract month.	`df["date"].dt.month`
.dt.day	Extract day.	`df["date"].dt.day`
.dt.dayofweek	0=Monday … 6=Sunday.	`df["date"].dt.dayofweek`
.dt.isocalendar().week	ISO week number.	`df["date"].dt.isocalendar().week`
.dt.quarter	Quarter.	`df["date"].dt.quarter`
pd.Timedelta	Represent time differences.	`pd.Timedelta(days=7)`
pd.date_range()	Generate date sequence.	`pd.date_range("2026-01-01", periods=7)`
resample()	Time-based aggregation.	`df.set_index("date").resample("M")["revenue"].sum()`

Common pattern: convert first → validate timezone/format → use `.dt` → calculate differences → filter/group/resample.

9. Sorting & Ranking

Method / Pattern	What it does	Example
sort_values()	Sort by one/more columns.	`df.sort_values(["country","revenue"], ascending=[True,False])`
sort_index()	Sort by index.	`df.sort_index()`
rank()	Assign ranks.	`df["revenue"].rank(ascending=False, method="dense")`

Ranking methods: average, min, max, first, dense. SQL analogues: ROW_NUMBER, RANK, DENSE_RANK.

10. GroupBy — Split → Apply → Combine

Method / Pattern	What it does	Example
groupby()	Split data into groups.	`df.groupby("advisor_id")`
sum()	Sum per group.	`df.groupby("advisor_id")["revenue"].sum()`

Method / Pattern	What it does	Example
mean()	Mean per group.	`df.groupby("advisor_id")["revenue"].mean()`
count()	Non-null observations.	`df.groupby("advisor_id")["revenue"].count()`
size()	Rows in each group, including nulls.	`df.groupby("advisor_id").size()`
min()/max()	Group extrema.	`df.groupby("risk_level")["revenue"].max()`
median()	Group median.	`df.groupby("risk_level")["revenue"].median()`
std()/var()	Dispersion.	`df.groupby("risk_level")["revenue"].std()`
nunique()	Unique count per group.	`df.groupby("advisor_id")["customer_id"].nunique()`
agg()	Multiple aggregations.	`df.groupby("advisor_id")["revenue"].agg(["sum","mean","max"])`
Named aggregation	Readable output names.	`df.groupby("advisor_id").agg(avg_rev=("revenue","mean"))`
transform()	Group result aligned back to original rows.	`df["advisor_avg"] = df.groupby("advisor_id")["revenue"].transform("mean")`
filter()	Keep/drop entire groups by a condition.	`df.groupby("advisor_id").filter(lambda g: len(g) >= 10)`

Key distinction: `agg()` reduces groups; `transform()` preserves the original row count; `filter()` decides whether whole groups remain.

11. Merge / JOIN / Concat

Method / Pattern	What it does	Example
pd.merge()	SQL-like relational join.	`pd.merge(customers, orders, on="customer_id", how="left")`
how="inner"	Only matching keys.	`pd.merge(a,b,on="id",how="inner")`
how="left"	Keep all left rows.	`pd.merge(a,b,on="id",how="left")`
how="right"	Keep all right rows.	`pd.merge(a,b,on="id",how="right")`
how="outer"	Keep all keys from both.	`pd.merge(a,b,on="id",how="outer")`
how="cross"	Cartesian product.	`pd.merge(a,b,how="cross")`
left_on/right_on	Join differently named keys.	`pd.merge(a,b,left_on="cust_id",right_on="id")`
suffixes	Disambiguate overlapping columns.	`pd.merge(a,b,on="id",suffixes=("_left","_right"))`
pd.concat()	Stack/align objects along an axis.	`pd.concat([df1, df2], axis=0, ignore_index=True)`
DataFrame.join()	Convenient index-based join.	`df1.join(df2, how="left")`

Merge vs concat: merge combines records based on keys; concat appends/alines objects along an axis. **SQL JOIN ≈ pd.merge().**

12. Reshaping & Cross-Tabulation

Method / Pattern	What it does	Example
pivot()	Long → wide when combinations are unique.	`df.pivot(index="date", columns="product", values="revenue")`
pivot_table()	Pivot with aggregation; handles duplicates.	`df.pivot_table(index="country", columns="risk_level", values="revenue", aggfunc="mean")`
melt()	Wide → long.	`df.melt(id_vars=["customer_id"], var_name="metric", value_name="value")`
stack()	Move columns into an index level.	`df.stack()`
unstack()	Move index level into columns.	`df.unstack()`
pd.crosstab()	Frequency/aggregation table for categorical variables.	`pd.crosstab(df["country"], df["risk_level"])`

13. map / apply / Vectorization / Lambda

Method / Pattern	What it does	Example
Series.map()	Element-wise mapping on a Series.	`df["risk_level"].map({"Low":1,"Medium":2,"High":3})`
DataFrame.map()	Element-wise operation over DataFrame cells (modern replacement for applymap).	`df[cols].map(lambda x: x.strip() if isinstance(x,str) else x)`
Series.apply()	Apply Python function to Series values.	`df["age"].apply(lambda x: "Senior" if x > 60 else "Adult")`
DataFrame.apply()	Apply function across rows/columns.	`df[["revenue","age"]].apply("mean")`
np.where()	Vectorized binary condition.	`np.where(df["age"] >= 18, "Adult", "Minor")`
np.select()	Vectorized multi-condition logic.	`np.select([df["age"]<18, df["age"]<=60], ["Minor","Adult"], default="Senior")`

Performance rule: prefer vectorized Pandas/NumPy operations over Python loops and excessive `apply()`. Use `apply()` when the transformation genuinely needs a Python function.

14. Conditional Transformation

Method / Pattern	What it does	Example
np.where()	Two-way condition.	`np.where(df["age"] < 18, "Minor", "Adult")`
np.select()	Multiple conditions.	`np.select(conditions, choices, default="Other")`
where()	Keep values where condition is true; replace false.	`df["revenue"].where(df["revenue"] >= 0, 0)`
mask()	Replace values where condition is true.	`df["revenue"].mask(df["revenue"] < 0, 0)`
loc	Explicit conditional assignment.	`df.loc[df["age"] < 18, "segment"] = "Minor"`

15. Window-Like / Analytical Operations

Method / Pattern	What it does	Example
shift()	Previous/next row value.	`df["prev_rev"] = df["revenue"].shift(1)`
diff()	Difference from prior row.	`df["delta"] = df["revenue"].diff()`
pct_change()	Percentage change.	`df["growth"] = df["revenue"].pct_change()`
rolling()	Moving-window calculations.	`df["ma7"] = df["revenue"].rolling(7).mean()`
expanding()	Cumulative expanding-window calculations.	`df["running_avg"] = df["revenue"].expanding().mean()`
cumsum()	Running total.	`df["running_total"] = df["revenue"].cumsum()`
cummax()	Running maximum.	`df["peak"] = df["revenue"].cummax()`
cummin()	Running minimum.	`df["floor"] = df["revenue"].cummin()`
cumprod()	Cumulative product.	`df["factor"] = df["factor"].cumprod()`
rank()	Analytical ranking.	`df["rank"] = df["revenue"].rank(method="dense", ascending=False)`

16. Index & MultiIndex

Method / Pattern	What it does	Example
set_index()	Use column(s) as index.	`df.set_index("customer_id")`
reset_index()	Move index back to columns.	`df.reset_index()`

Method / Pattern	What it does	Example
reindex()	Conform to a new label set.	`df.reindex(new_index)`
rename_axis()	Name index/column axes.	`df.rename_axis("customer_id")`
MultiIndex	Hierarchical index with multiple levels.	`df.set_index(["country","risk_level"])`
loc	Select hierarchical labels.	`df.loc[("India","High")]`
stack()/unstack()	Reshape MultiIndex levels.	`df.unstack()`

Guideline: indexes are powerful for alignment and time-series work, but don't force a complex MultiIndex when ordinary columns make the data model clearer.

17. Data Validation Patterns

Method / Pattern	What it does	Example
Null primary key	Detect missing IDs.	`df["customer_id"].isna().any()`
Duplicate ID	Detect duplicate business keys.	`df["customer_id"].duplicated().any()`
Allowed categories	Validate domain values.	`df["risk_level"].isin(["Low","Medium","High"]).all()`
Numeric range	Validate numeric bounds.	`df["age"].between(18,100).all()`
Date validity	Check parse failures/order.	`df["completion_date"] >= df["onboarding_date"]`
Referential integrity	Find unmatched foreign keys.	`orders["customer_id"].isin(customers["customer_id"])`
Unexpected nulls	Count missingness by column.	`df.isna().sum()`
Data type validation	Check schema.	`df.dtypes`

18. Pandas ETL Pattern

```
Extract → Read CSV / API / SQL / Parquet
Transform → Clean → type conversion → joins → features → aggregates
Validate → keys → nulls → ranges → categories → referential integrity
Load → CSV / Parquet / SQL / downstream system
```

Production principle: make transformations deterministic, validate before loading, log row counts and rejected records, and avoid silently coercing bad data without tracking it.

19. File Formats & Performance

Method / Pattern	What it does	Example
CSV	Portable, human-readable, row-oriented; larger/slower for analytics.	`pd.read_csv("data.csv")`
Parquet	Columnar, typed, compressed; preferred for analytical data lakes.	`pd.read_parquet("data.parquet")`
usecols	Read only needed CSV columns.	`pd.read_csv("x.csv", usecols=["id","revenue"])`
dtype	Control dtypes at ingestion.	`pd.read_csv("x.csv", dtype={"id":"string"})`
chunksize	Process large CSV incrementally.	`for chunk in pd.read_csv("x.csv", chunksize=50000): ...`
category	Compact repeated categorical strings.	`df["country"] = df["country"].astype("category")`
vectorization	Use array-level operations instead of Python loops.	`df["total"] = df["price"] * df["qty"]`
memory_usage	Measure memory footprint.	`df.memory_usage(deep=True).sum()`

Pandas limitation: it is primarily in-memory. For data larger than available memory or distributed workloads, consider chunking, SQL pushdown, Polars, or PySpark depending on the problem.

20. Pandas ↔ SQL Quick Translation

Method / Pattern	What it does	Example
SELECT	Select columns.	`df[["name","revenue"]]`
WHERE	Filter rows.	`df[df["revenue"] > 10000]`
DISTINCT	Unique values.	`df["country"].drop_duplicates()`
GROUP BY	Group and aggregate.	`df.groupby("country")["revenue"].sum()`
HAVING	Filter aggregated groups.	`g=df.groupby("country")["revenue"].sum(); g[g>100000]`
ORDER BY	Sort.	`df.sort_values("revenue", ascending=False)`
JOIN	Relational join.	`pd.merge(a,b,on="customer_id",how="left")`
CASE WHEN	Conditional expression.	`np.select([df["age"]<18,df["age"]<=60],["Minor","Adult"],default="Senior")`
ROW_NUMBER	Sequential rank.	`df["rn"] = df.groupby("advisor_id").cumcount()+1`
RANK	Competition/average rank.	`df["rank"] = df["revenue"].rank(method="min", ascending=False)`

21. Pandas ↔ PySpark Decision Rule

Pandas: local/in-memory analysis, moderate data, rich interactive exploration. **PySpark:** distributed processing, very large datasets, cluster/cloud ETL. **SQL:** push transformations to the database when the data already lives there and the database is the right execution engine.

22. High-Value Real-World Patterns

Method / Pattern	What it does	Example
KPI total	Count records.	`df["customer_id"].nunique()`
Completion rate	Completed / total.	`(df["status"].eq("Completed").mean())`
Average duration	Date difference then mean.	`(df["completion_date"]-df["onboarding_date"]).dt.days.mean()`
Top advisor	Aggregate then sort.	`df.groupby("advisor_id")["revenue"].sum().sort_values(ascending=False)`
Monthly volume	Convert date and group by period.	`df.groupby(df["onboarding_date"].dt.to_period("M")).size()`
Null profile	Missing count and percentage.	`df.isna().agg(["sum","mean"]).T`
Duplicate business key	Duplicate IDs.	`df[df["customer_id"].duplicated(keep=False)]`
Conditional segment	Multi-condition feature.	`np.select([df["age"]<30,df["age"]<60],["Young","Adult"],default="Senior")`
Group average on each row	Use transform.	`df["advisor_avg"] = df.groupby("advisor_id")["revenue"].transform("mean")`
Previous row	Use shift after correct sorting.	`df = df.sort_values("date"); df["prev"] = df["revenue"].shift()`

23. Interview Traps to Remember

- **`df["col"]` vs `df[["col"]]`:** Series vs DataFrame.
- **`count()` vs `size()`:** count ignores nulls in the counted column; size counts rows in the group.
- **`agg()` vs `transform()`:** agg reduces; transform preserves original row count.
- **`merge()` vs `concat()`:** merge uses relational keys; concat combines along an axis.
- **`loc` vs `iloc`:** labels vs integer positions.
- **`where()` vs `mask()`:** where keeps values where condition is true; mask replaces values where condition is true.

• **`isna()` vs `dropna()`:** detect vs remove missing values.

• **`pivot()` vs `pivot_table()`:** pivot requires unique index/column combinations; pivot_table aggregates duplicates.

• **`count()` vs `nunique()`:** observations vs distinct values.

• **Sort before `shift()`/`diff()`:** row order must represent the intended sequence.

• **Avoid chained assignment:** prefer explicit `.loc[...] = ...` to make mutation unambiguous.

• **Do not blindly `fillna(0)`:** zero and unknown are different business meanings.

24. One-Page Revision Checklist

Core: Series, DataFrame, Index, Columns, Values, Axis
I/O: read_csv, read_excel, read_json, read_sql, read_parquet, to_csv, to_excel, to_json, to_sql, to_parquet
Inspect: head, tail, sample, shape, size, columns, index, dtypes, info, describe, nunique, value_counts, memory_usage
Select: [], [[]], loc, iloc, at, iat, isin, between, query
Transform: assignment, assign, loc, iloc, rename, drop, replace, mask, where
Clean: isna, notna, dropna, fillna, ffill, bfill, duplicated, drop_duplicates, astype, to_numeric, to_datetime
Strings: lower, upper, title, strip, replace, contains, startswith, endswith, len, split, extract, slice, match
Dates: to_datetime, dt.year/month/day/dayofweek/quarter, date_range, Timedelta, resample
GroupBy: sum, mean, count, size, min, max, median, std, var, nunique, agg, named aggregation, transform, filter
Joins: merge inner/left/right/outer/cross, on, left_on, right_on, suffixes, join
Reshape: pivot, pivot_table, melt, stack, unstack, crosstab
Apply: map, apply, DataFrame.map, vectorization, lambda, np.where, np.select
Windows: shift, diff, pct_change, rolling, expanding, cumsum, cummax, cummin, cumprod, rank
Index: set_index, reset_index, reindex, rename_axis, MultiIndex
Validation: null keys, duplicates, ranges, categories, dates, referential integrity, dtypes
Performance: vectorize, select required columns, dtype optimization, category, chunksize, memory_usage, avoid excessive apply/loops
Data Engineering: ETL, SQL pushdown, Parquet, validation, logging, rejected records, Pandas vs SQL vs PySpark